

코드 컨벤션과 패키지 관리

파이썬 코드 컨벤션과 패키지 관리를 위한 uv

파이썬 코드 컨벤션

파이썬 코드 컨벤션이란

- 파이썬 코드를 일관되고 읽기 쉽게 작성하기 위한 규칙
- 대표적으로 들여쓰기, 변수명, 함수명, 공백, 줄 길이, import 순서 같은 작성 방식을 포함
- 코드 컨벤션을 지키면 다른 사람이 코드를 더 쉽게 이해하고 유지보수할 수 있음

 **Bad**

```
def f(x,y):  
    return x+y
```

 **Good**

```
def add_numbers(first_number, second_number):  
    return first_number + second_number
```

 명확한 이름

 일관된 공백

 읽기 쉬운 구조

파이썬 코드 컨벤션의 중요성



가독성

코드를 쉽게 읽고 빠르게 이해할 수 있음



유지 보수성

변경이 쉬워지고 버그를 줄일 수 있음



협업 생산성

팀원 간 원활한 소통과 작업 속도 향상



일관성

프로젝트 전반의 코드 품질을 일정하게 유지 가능

PEP 8 핵심 규칙

- 들여쓰기: 4칸 사용
- 줄 길이: 79자 이내 권장
- 공백 규칙: 연산자 주변, 콤마 뒤 공백
- **import** 정리: 표준 -> 서드파티 -> 로컬
- 빈 줄 사용: 가독성을 위해 적절히 추가

코드 예시

python

```
import os
import numpy as np

def add(a, b):
    """두 수의 합을 반환"""
    return a + b

def mean(values):
    total = sum(values)
    return total / len(values)
```

좋은 이름 짓기



변수명: 의미가 명확하고 구체적으로



함수명: 동사 + 목적어 형태로



클래스명: 명사(파스칼 케이스) 사용



상수명: 대문자 + 언더스코어



불리언 변수명: is/has/can 등으로 시작

코드 예시

```
python
model, tokenizer          # 변수명

load_data(), train_model() # 함수명

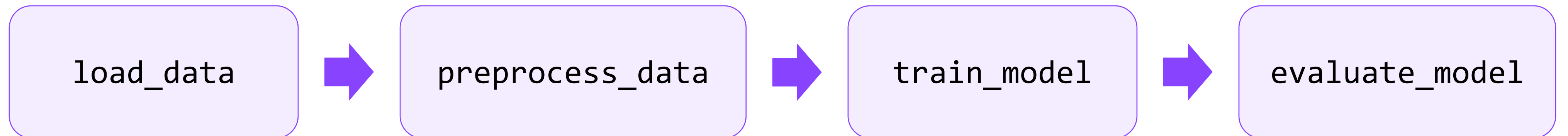
TextClassifier, DataModule # 클래스명

MAX_LEN, LEARNING_RATE    # 상수명

is_training, has_padding  # 불리언 변수명
```

함수 작성 컨벤션

- **한 함수 = 한 역할**: 단일 책임 원칙
- **인자 수 줄이기**: 단순하고 명확하게
- **반환값 명확화**: 예측 가능한 결과 제공
- **중복 제거**: 재사용 가능한 함수 만들기
- **테스트 가능성**: 작은 단위로 나누기



주석과 문서화

주석은 이유를 설명: '무엇' 보다 '왜'에 집중

self-explanatory code: 자연스럽게 이해되는 코드

docstring 작성: 함수/모듈 목적과 사용법

공개 함수 문서화: 외부 사용자를 고려

파라미터/반환값 설명: 타입과 의미 명시

코드 예시

python

```
def tokenize(text: str) -> list[str]:  
    """텍스트를 토큰 리스트로 변환하다.  
    Args:  
        text (str): 입력 텍스트  
    Returns:  
        list[str]: 토큰 리스트  
    """  
    ...
```

타입 힌트

기본 타입: int, float, str, bool

list/dict: list[T], dict[K, V]

Optional: Optional[T] 또는 T | None

반환 타입: -> T 로 표현

협업 도움: IDE 지원, 버그 감소

코드 예시

```
python
from typing import Optional

def predict(texts: list[str]) -> list[str]:
    ...

def get_threshold() -> Optional[float]:
    ...

def is_valid(score: float) -> bool:
    ...
```

예외 처리 컨벤션

구체적 예외 사용: 예외 타입을 명확히

bare except 지양: 원인 파악이 어려움

예외 메시지 명확화: 맥락이 드러나게

복구 가능한 경우만 처리: 불필요한 예외 숨김 방지

로깅 고려: 문제 추적과 디버깅에 도움



파일 로딩 실패

`FileNotFoundError`



API 호출 실패

`requests.RequestException`



추론 오류

`RuntimeError, ValueError`

프로젝트 구조와 모듈 분리

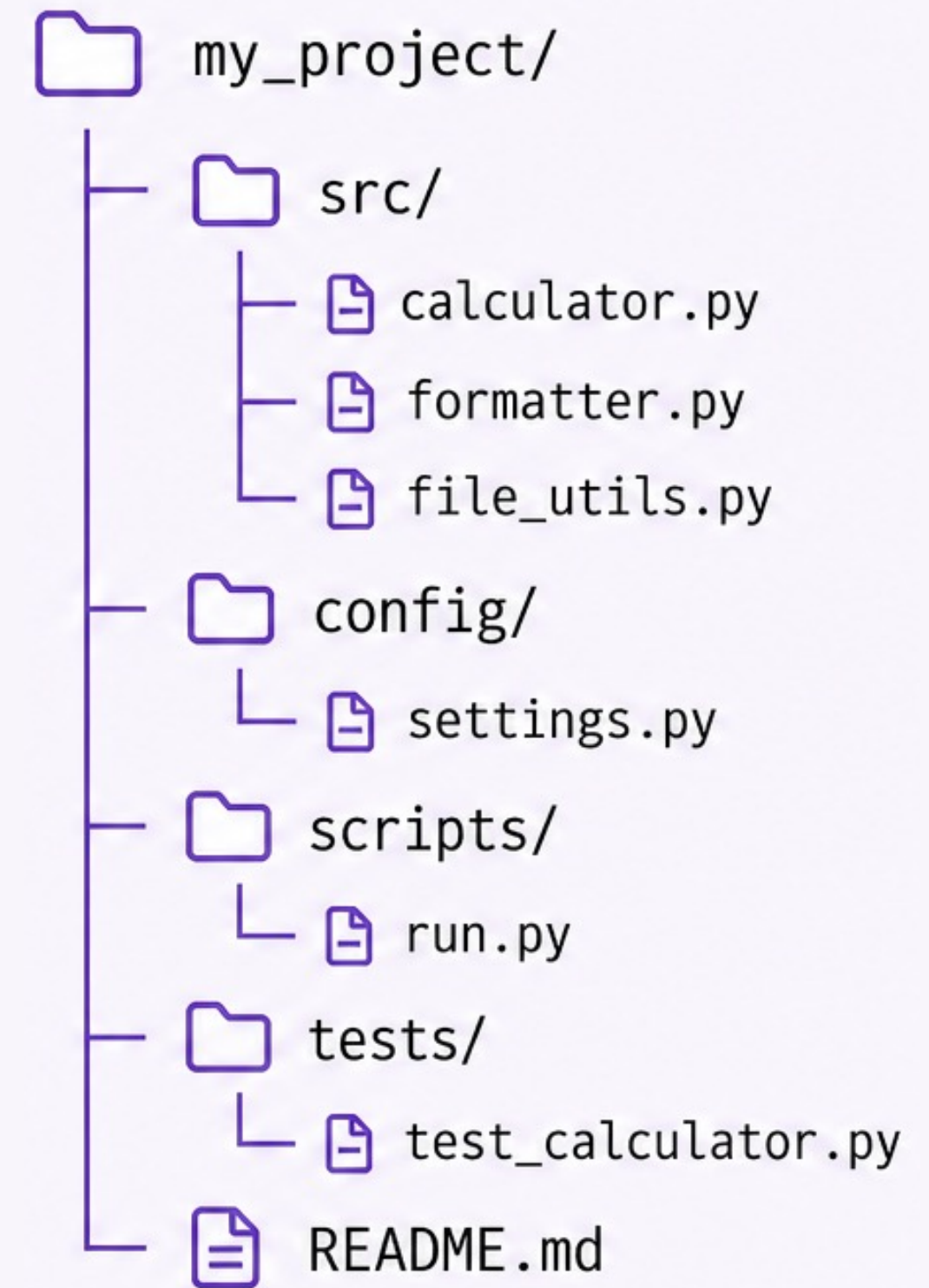
역할별 파일 분리: 기능에 따라 코드를 나누어 관리

공통 기능 모듈화: 반복되는 코드를 별도 파일로 분리

실행 코드 최소화: 실행 흐름은 단순하고 명확하게

설정값 분리: 경로, 옵션, 상수는 한곳에서 관리

테스트 코드 분리: 기능 검증 코드를 별도로 작성



포매퍼와 린터

Black: 코드 포매팅 (스타일 통일)

Ruff: 린팅 + 일부 자동 수정

isort: import문 정렬

코드 리뷰 효율 향상

저장 시 자동 실행 설정 권장

자동 정리 워크플로우



코드 작성
(Write code)



포매퍼 (Black)
자동 스타일 정리



린터 (Ruff)
문제 감지/수정 제안



깨끗한 코드
(Clean code)

AI 코드 작성 시 유의사항

- AI는 대부분의 경우 일반적인 코드 컨벤션을 기반으로 코드를 생성함
- 반복적인 수정 요청과 기능 확장 과정에서 코드 스타일의 일관성이 점차 저하될 수 있음
- 따라서 AI가 생성한 코드라도 최종적으로는 **개발자가 코드 컨벤션을 이해하고 검토**할 수 있어야 함

1



하드코딩된
경로/하이퍼파라미터

2



전역 변수 남용

3



긴 함수
한 함수가
너무 많은 역할

4



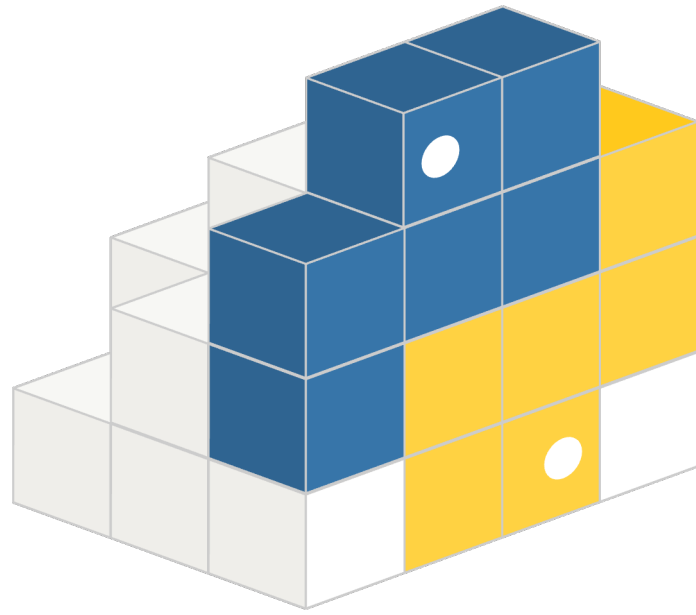
중복 코드

5



의미 없는 변수명

pip를 활용한 설치 방법



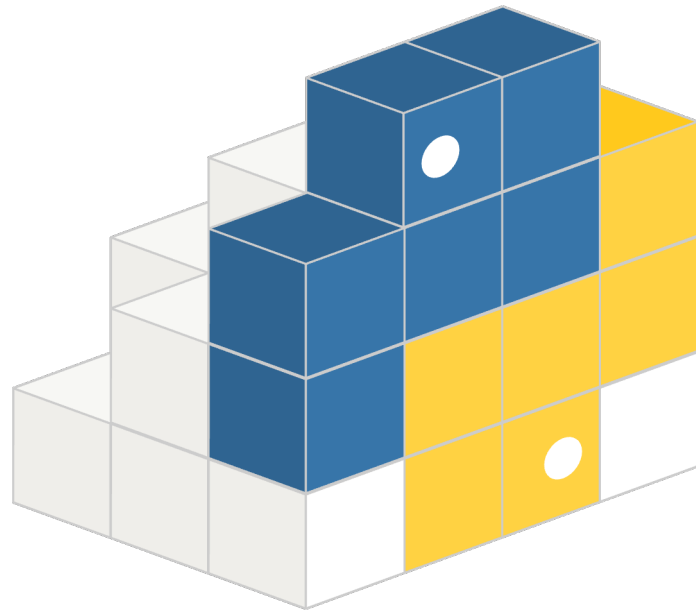
PyPI

= 파이썬 패키지를 호스팅하고 배포하는 공식 저장소

그렇다면, PyPI에 공유된 패키지는 어떻게 설치할 수 있을까?



정답은 pip!



pip

- PyPI에 공유된 패키지들이 저장된 저장소에서 패키지를 검색하고 설치
- pip를 활용하여 특정 패키지 설치, 업그레이드, 제거, 패키지 목록 관리 등이 가능



pip 설치 관련 주요 명령어

설치하기	pip install <패키지명>
특정 버전의 패키지 설치하기	pip install <패키지명>==버전
설치된 전체 패키지 & 버전 확인	pip list
설치된 특정 패키지 & 버전 확인	pip list grep <패키지명>
버전 업그레이드하기	pip install --upgrade <패키지명>
설치된 패키지 제거하기	pip uninstall <패키지명>



설치 과정 살펴보기

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash + v [ ] [ ] ... ^ X

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

o elicer@4b8b61724834:~$ [ ]
```

명시하지 않은 패키지들이 같이 설치되는 이유 = pip의 의존성 관리 기능



설치 과정 살펴보기

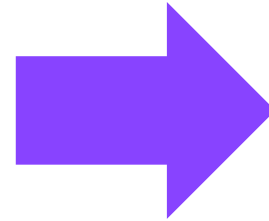
```
Collecting charset-normalizer<4,>=2
  Downloading charset_normalizer-3.3.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
  (142 kB)
Collecting certifi<2024.4.17
```

명시하지 않은 패키지들이 같이 설치되는 이유 = pip의 의존성 관리 기능

만약, 설치해야 할 패키지들이 많을 경우?

장보기 목록

우유 1.8L 2개
유정란 30구 3개
고기 만두 100g 5개
...



requirements.txt

```
aiofiles==23.2.1
aiohttp==3.8.4
aiosignal==1.3.1
alabaster==0.7.12
altair==5.1.2
altgraph==0.17.4
amqp==5.1.1
anaconda-client==1.9.0
anaconda-project==0.10.2
annotated-types== 0.6.0
anyio==3.7.1
appdirs==1.4.4
appnope==0.1.2
appscript==1.1.2
Automat==20.2.0
Babel==2.9.1
backcall==0.2.0
backoff==2.2.1
backports.functools-lru-cache==1.6.4
backports.tempfile==1.0
backports.weakref==1.0.post1
bcrypt==3.2.0
beautifulsoup4==4.11.1
billiard==3.6.4.0
```



requirements.txt

설치해야 할 패키지들과 버전 목록을 명시한 파일

requirements.txt

```
aiofiles==23.2.1
aiohttp==3.8.4
aiosignal==1.3.1
alabaster==0.7.12
altair==5.1.2
altgraph==0.17.4
amqp==5.1.1
anaconda-client==1.9.0
anaconda-project==0.10.2
annotated-types== 0.6.0
anyio==3.7.1
appdirs==1.4.4
appnope==0.1.2
appscript==1.1.2
Automat==20.2.0
Babel==2.9.1
backcall==0.2.0
backoff==2.2.1
backports.functools-lru-cache==1.6.4
backports.tempfile==1.0
backports weakref==1.0.post1
bcrypt==3.2.0
beautifulsoup4==4.11.1
billiard==3.6.4.0
```

pip install -r requirements.txt

-r : 해당 옵션 뒤에 오는 파일 경로를 읽어
해당 파일 내에 명시된 패키지를 설치



requirements.txt 생성하기

내 환경에서 설치된 패키지 목록과 버전을 한 번에 txt 파일로 내보낼 수 있음

requirements.txt

```
aiofiles==23.2.1
aiohttp==3.8.4
aiosignal==1.3.1
alabaster==0.7.12
altair==5.1.2
altgraph==0.17.4
amqp==5.1.1
anaconda-client==1.9.0
anaconda-project==0.10.2
annotated-types== 0.6.0
anyio==3.7.1
appdirs==1.4.4
appnope==0.1.2
appscript==1.1.2
Automat==20.2.0
Babel==2.9.1
backcall==0.2.0
backoff==2.2.1
backports.functools-lru-cache==1.6.4
backports.tempfile==1.0
backports weakref==1.0.post1
bcrypt==3.2.0
beautifulsoup4==4.11.1
billiard==3.6.4.0
```

pip freeze > requirements.txt

현재 환경에 설치된 모든 패키지와 버전을 담은
requirements 라는 이름의 txt 파일 생성

uv를 활용한 설치 방법



가상환경 제작



pip install



패키지 목록 기록



가상환경 제작



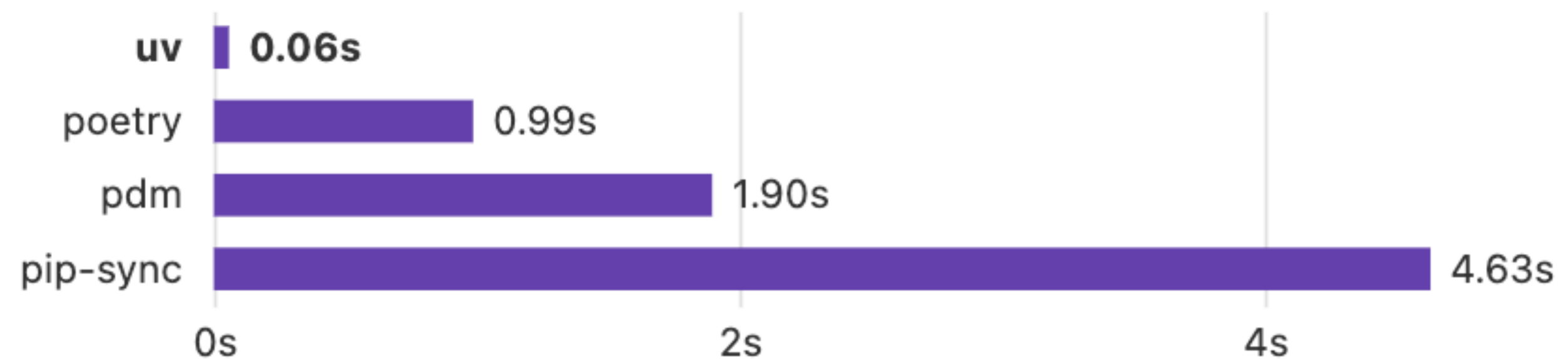
uv **pip** install



패키지 목록 기록

Rust로 만들어진 초고속 파이썬 패키지·프로젝트 관리 도구

An extremely fast Python package and project manager, written in Rust.



Installing Trio's dependencies with a warm cache.

pip 대비 10~100배 빠름

pip, venv 등 여러 도구 대체

pip와 호환



OpenAI, Astral 인수... Codex 개발자 도구 강화

Investing.com | 기사 편집 Vlad Schepkov | 기업 뉴스

입력: 2026- 03- 19- 오후 10:27

OpenAI to acquire Astral

ASTRAL

TOOLS ▾

DOCS ^

BLOG

COMPANY



RUFF

Lint at lightspeed



UV

All-in-one Python packaging



TY

A next-generation type checker



다양한 AI 프로젝트에서 환경 구성 시 uv를 권장



Model Context Protocol

Q Search...

⌘K

🌟 Ask Assistant

Documentation

Extensions

Specification

Registry

SEPs

Community

Get started

What is MCP?

About MCP

Architecture

Servers

Clients

Versioning

Develop with MCP

Connect to local MCP servers

macOS/Linux Windows



```
# Create a new directory for our project
```

```
uv init weather
```

```
cd weather
```

```
# Create virtual environment and activate it
```

```
uv venv
```

```
source .venv/bin/activate
```

```
# Install dependencies
```

```
uv add "mcp[cli]" httpx
```

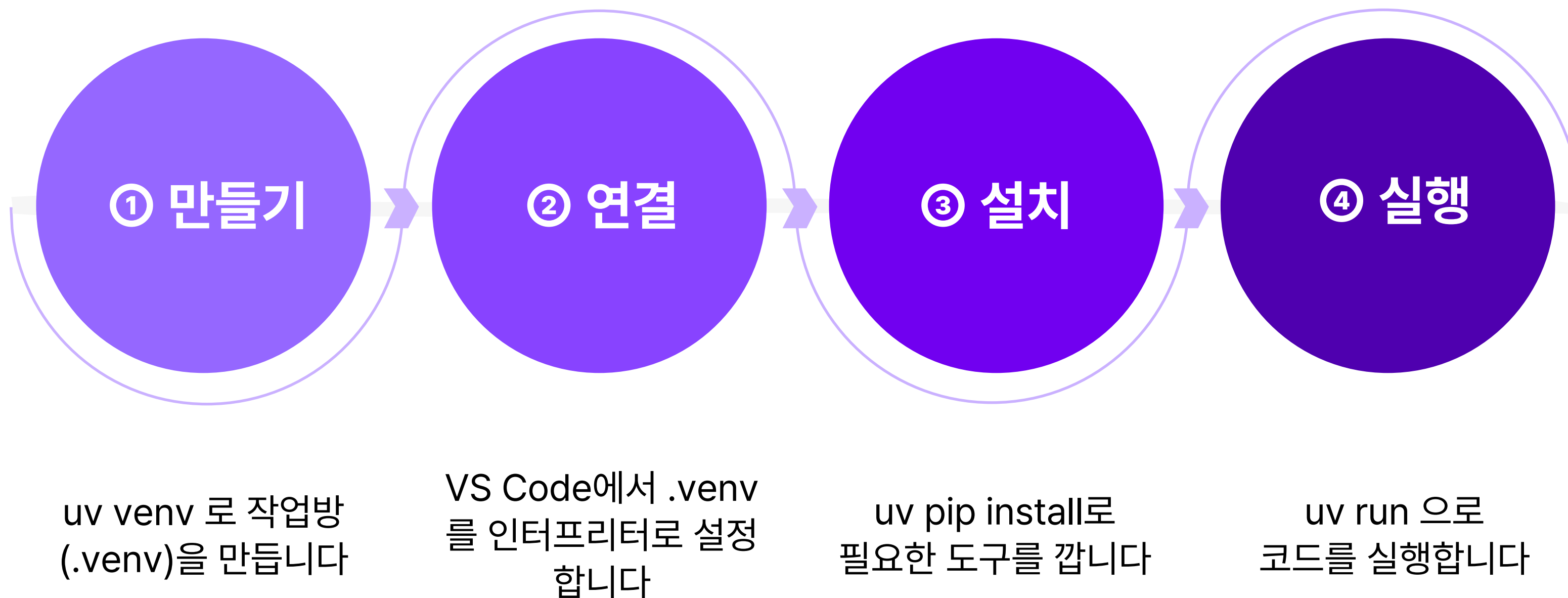
```
# Create our server file
```

```
touch weather.py
```



uv로 환경을 다루는 4단계

환경 만들기부터 실행까지, 명령 네 개



uv venv

현재 폴더에 .venv 라는 작업방을 만듭니다.

uv venv myenv 처럼 커스텀 이름을 지어줄 수도 있습니다.

uv venv --python 3.12

파이썬 버전을 지정하고 싶을 때

uv venv --seed

작업방 안에 pip를 같이 넣기

```
elicer@4675a8970c2d:~$ uv venv
Using CPython 3.12.9 interpreter at: /usr/local/bin/python3
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
```



```
▼ .venv
  > bin
  > lib
  > lib64
  ◆ .gitignore
  ≡ CACHEDIR.TAG
  ⚙ pyvenv.cfg
```

① Ctrl + Shift + P 혹은 Cmd + Shift + P 로 명령 팔레트 실행

② Select Interpreter 검색

>select interp

Python: **Select Interpreter**

Select to Bracket

similar commands

Select Interpreter

Selected Interpreter: `./venv/bin/python`

+ Create Virtual Environment...

Enter interpreter path

Python 3.12.9 ('.venv': venv) `./venv/bin/python`

Recommended

Python 3.12.9 64-bit `/usr/local/bin/python3`

Global

Python 3.10.12 64-bit `/bin/python3`

Python 3.10.12 64-bit `/usr/bin/python3`

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

o elicer@2d564d449e47:~/elice_uv\$

③ .venv 환경 선택

Python 3.12.9 ('.venv': venv)

④ 선택된 인터프리터 확인



uv pip install 패키지명

현재 작업방(.venv) 안에 패키지를 설치합니다.

```
uv pip install 패키지1 패키지2
```

띄어쓰기를 기준으로 여러 패키지를 한 번에 설치

```
uv pip install -r requirements.txt
```

txt 파일에 명시된 패키지 설치

```
uv pip install "pandas==2.2.0"
```

특정 버전을 집어 설치

```
uv pip uninstall 패키지명
```

해당 패키지 삭제

```
uv pip list
```

패키지 목록 확인



도구 설치

① 패키지 설치

```
elicer@4675a8970c2d:~$ uv pip install pandas
Resolved 4 packages in 653ms
Prepared 4 packages in 2.69s
Installed 4 packages in 808ms
+ numpy==2.5.0
+ pandas==3.0.4
+ python-dateutil==2.9.0.post0
+ six==1.17.0
```

③ 설치 결과 확인

```
elicer@4675a8970c2d:~$ uv pip list
Package            Version
-----
numpy              2.5.0
pandas             3.0.4
python-dateutil    2.9.0.post0
six               1.17.0
```

② site-packages에 생성



```
lib64 / python3.12 / site-packages
> dateutil
> numpy
> numpy-2.5.0.dist-info
> numpy.libs
> pandas
> pandas-3.0.4.dist-info
> python_dateutil-2.9.0.post0.dist-info
> six-1.17.0.dist-info
≡ _virtualenv.pth
🔗 _virtualenv.py
🔗 six.py
```

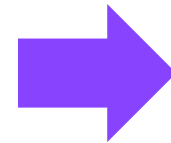
uv run 파일명

같은 폴더의 작업방(.venv)를 자동으로 찾아 해당 환경에서 파이썬 파일을 실행합니다.

main.py

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "이름": ["김분석", "이마케팅", "박운영"],
5     "부서": ["데이터", "마케팅", "운영"],
6     "연차": [3, 5, 2],
7 })
8
9 print(df)
10 print("\npandas 버전:", pd.__version__)
```

설치한 pandas 활용



```
eLicer@4675a8970c2d:~$ uv run main.py
      이름  부서  연차
0  김 분석  데 이 터    3
1  이 마 케 팅  마 케 팅    5
2  박 운 영  운 영    2

pandas 버전 : 3.0.4
```



함께 쓰려면, 환경을 '재현' 할 수 있어야 함

현재 방식

환경 넘길 때 뭘 깔았는지
일일이 알려줘야 함

버전이 달라
"나는 되는데 너는 안 돼"

새 PC에 처음부터
환경 만들기 번거로움

uv 프로젝트 방식

uv add로 설치 → pyproject.toml에
자동 기록, 파일만 공유하면 끝

uv.lock이 버전까지 고정 →
uv sync 하면 똑같이 복제

uv init으로 시작 → 작업방까지
자동, 단계가 줄어들음



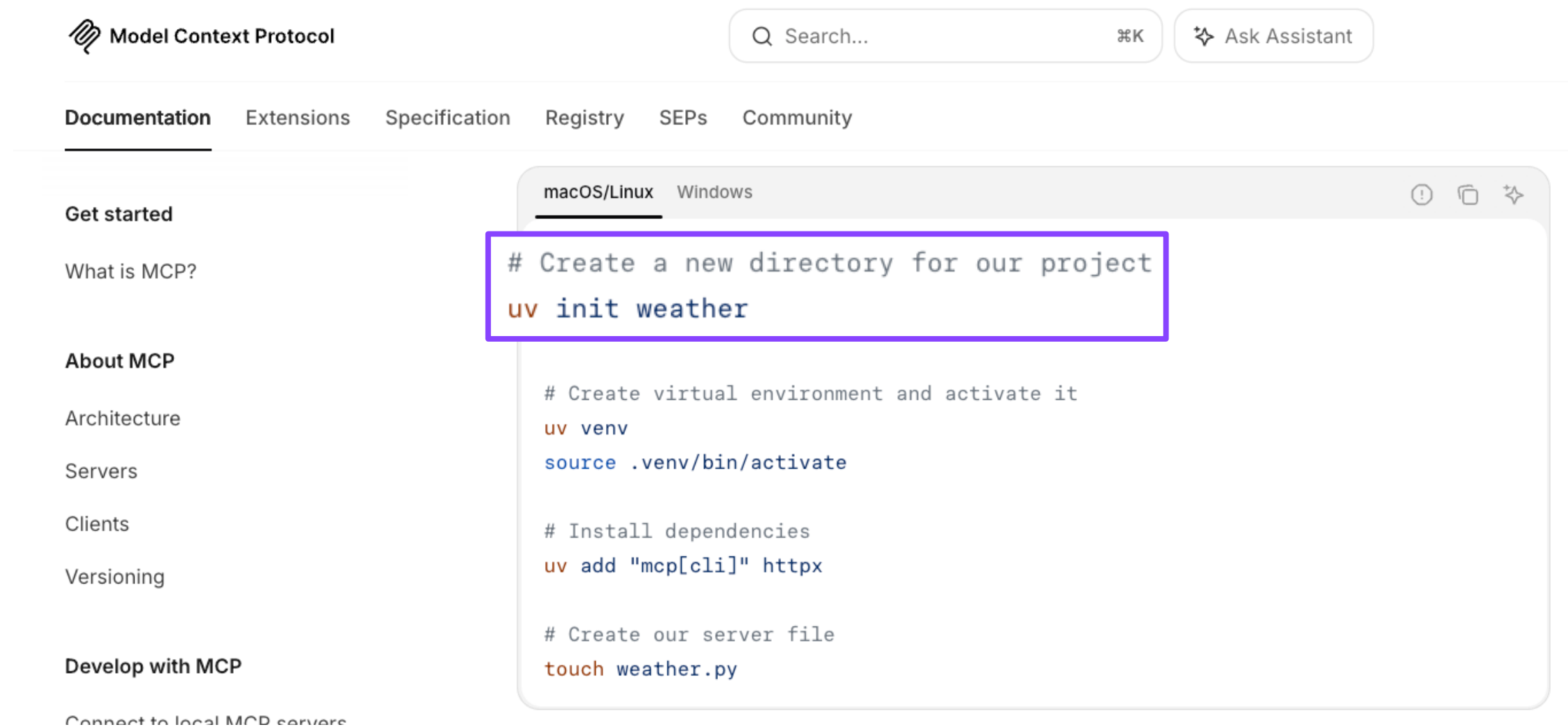
uv init

uv에서 현재 디렉토리에 새로운 파이썬 프로젝트를 초기화하는 명령어

자동으로 pyproject.toml, main.py, README.md, 가상환경을 생성합니다.

```
uv init my-project
```

새 디렉토리를 만들고 그 안에 초기화





uv init

```
elicer@4675a8970c2d:~/elice uv$ uv init
```

```
Initialized project `elice-uv`
```

▼ ELICE_UV

 main.py

 pyproject.toml

 README.md

 main.py

```
1  def main():
3
4
5  if __name__ == "__main__":
6      |    main()
7
```



uv init

```
elicer@4675a8970c2d:~/elice uv$ uv init
```

```
Initialized project `elice-uv`
```

▼ ELICE_UV

 main.py

 pyproject.toml

 README.md

⚙ pyproject.toml

```
1 [project]
2 name = "elice-uv"
3 version = "0.1.0"
4 description = "Add your description here"
5 readme = "README.md"
6 requires-python = ">=3.12"
7 dependencies = []
```



uv init

```
elicer@4675a8970c2d:~/elice uv$ uv init
```

```
Initialized project `elice-uv`
```

▼ ELICE_UV

 main.py

 pyproject.toml

 README.md

 README.md

1

uv add 패키지명

단순히 패키지를 설치하는 것이 아니라, 아래 3가지 동작을 함께 수행

- ① pyproject.toml 파일에 패키지를 기록
- ② 의존성을 해석해서 uv.lock 파일을 갱신
- ③ 가상환경에 실제로 패키지를 설치

```
uv add 패키지1 패키지2
```

띄어쓰기를 기준으로 여러 패키지를 한 번에 설치

```
uv remove 패키지명
```

해당 패키지 삭제

```
uv add "pandas==2.2.0"
```

특정 버전을 집어 설치

```
uv tree
```

설치된 패키지들간의 의존성 확인



uv 설치, 두 가지 방식

같은 .venv를 쓰지만 의존성을 '기록·재현'하는 방식이 다르다

pip 호환 방식

기존 pip 사용법 그대로

uv venv

uv pip install <pkg>

uv pip freeze > ...

VS

프로젝트 방식 · 권장

선언적 · 자동 관리

uv init

uv add <pkg>

uv sync

의존성 기록	requirements.txt (수동)
버전 고정	약함
환경 복제	pip install -r
적합한 상황	간단한 작업 · pip 이전

pyproject.toml (자동)
uv.lock으로 고정
uv sync
협업 · 배포



패키지 관리, 이렇게 바뀝니다.

직접 관리하던 패키지 목록 — 이제 uv가 자동으로 기록·고정



requirements.txt

- pip freeze로 직접 생성
- 만든 시점의 스냅샷
- 직접·전이 의존성 뒤섞임



uv add



pyproject.toml + uv.lock

- ✓ uv add 시 자동 기록
- ✓ uv.lock이 버전까지 고정
- ✓ 직접 의존성과 lock 분리

목록을 직접 챙기던 일을, uv가 설치와 동시에 대신합니다



uv add 패키지명

```
gear pyproject.toml
1  [project]
2  name = "elice-uv"
3  version = "0.1.0"
4  description = "Add your description here"
5  readme = "README.md"
6  requires-python = ">=3.12"
7  dependencies = []
```

```
elicer@a4838c06204f:~/elice_uv$ uv add pandas
Resolved 7 packages in 355ms
Checked 4 packages in 0.34ms
```

```
gear pyproject.toml
1  [project]
2  name = "elice-uv"
3  version = "0.1.0"
4  description = "Add your description here"
5  readme = "README.md"
6  requires-python = ">=3.12"
7  dependencies = [
8      "pandas>=3.0.4",
9  ]
```

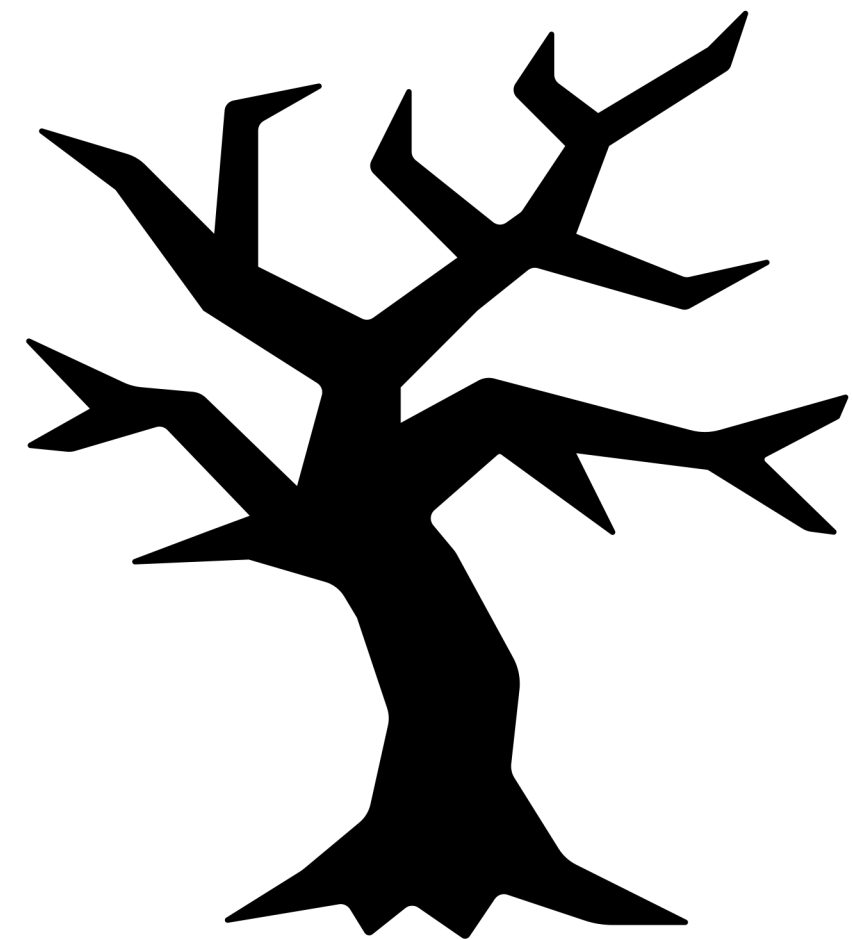
dependencies에
pandas가 추가된 모습



uv tree

각 의존성들을 tree (나무) 모양의 계층 형태로 보여줍니다.

```
elicer@a4838c06204f:~/elice_uv$ uv tree
Resolved 7 packages in 1ms
elicer v0.1.0
elice-uv v0.1.0
├─ pandas v3.0.4
│   └─ numpy v2.5.0
│       └─ python-dateutil v2.9.0.post0
│           └─ six v1.17.0
```





uv lock

pyproject.toml에 선언된 프로젝트 의존성을 해석(resolve)해서,
정확히 고정된 버전들을 uv.lock 파일에 기록합니다.

```
elicer@a4838c06204f:~/elice_uv$ uv lock
Using CPython 3.12.9 interpreter at: /usr/local/bin/python3
Resolved 6 packages in 130ms
```

▼ ELICE_UV

🔗 main.py

⚙️ pyproject.toml

📖 README.md

≡ uv.lock



패키지 고정

uv lock

≡ uv.lock

```
1  version = 1
2  revision = 3
3  requires-python = ">=3.12"
4  resolution-markers = [
5      "python_full_version >= '3.14' and sys_platform == 'win32'",
6      "python_full_version >= '3.14' and sys_platform == 'emscripten'",
7      "python_full_version >= '3.14' and sys_platform != 'emscripten' and sys_platform != 'win32'",
8      "python_full_version < '3.14' and sys_platform == 'win32'",
9      "python_full_version < '3.14' and sys_platform == 'emscripten'",
10     "python_full_version < '3.14' and sys_platform != 'emscripten' and sys_platform != 'win32'",
11 ]
12
13 [[package]]
14 name = "elice-uv"
15 version = "0.1.0"
16 source = { virtual = "." }
17 dependencies = [
18     { name = "pandas" },
19 ]
20
21 [package.metadata]
22 requires-dist = [{ name = "pandas", specifier = ">=3.0.4" }]
23
24 [[package]]
```

```
24 [[package]]
25 name = "numpy"
26 version = "2.5.0"
27 source = { registry = "https://pypi.org/simple" }
28 sdist = { url = "https://files.pythonhosted.org/packages/e7/05/3d27272d30698dc0ecb7fdfaa41ad70303b444f81722bb9!"
29 wheels = [
30     { url = "https://files.pythonhosted.org/packages/fa/0a/11486d02add7b1384dff7374d124b1cfbb0ee864dcc9f6a2c03!"
31     { url = "https://files.pythonhosted.org/packages/55/b2/285f48640a181947b4587a3766d21ec1eaa7fea833d4b49957e!"
32     { url = "https://files.pythonhosted.org/packages/dd/67/b032db1eb03ca30d16eda3b0c22aaa615338b9263c2fd559d0f!"
33     { url = "https://files.pythonhosted.org/packages/b9/83/03fc7300c7c6b6c84c487b1dc80d322817b95fbd1f4dd57a85e!"
34     { url = "https://files.pythonhosted.org/packages/82/49/2ec21730bc63ccfda829323f7040a8ed4715b3852ce658689cf!"
35     { url = "https://files.pythonhosted.org/packages/bb/6b/f4a3d0637692c49da8ef99d72d52526f92e0a8d6ac4f0ca9f31!"
36     { url = "https://files.pythonhosted.org/packages/3a/2f/c354ec86d1f3f5c19649463b0d39652e160736e5b0a4cd18dff!"
37     { url = "https://files.pythonhosted.org/packages/06/34/43efdc319988648580f93c11f1ae82cf7e2faa74925e98e454!"
38     { url = "https://files.pythonhosted.org/packages/71/e2/f5d1676b1d7fb682eb5e9a1641e7ebd2414b3216c370661d102!"
39     { url = "https://files.pythonhosted.org/packages/8f/7c/48f115d1c58a34032facebcd51fdf2d02df2c51d4a46a81dd11!"
40     { url = "https://files.pythonhosted.org/packages/86/26/2e0882f4044d1b1a1b63e875151fb2393389032022a8b7f5657!"
41     { url = "https://files.pythonhosted.org/packages/8a/33/07675aaad7f26ea013d5e884d9a0d784b79c6bd7566c333f5a5!"
42     { url = "https://files.pythonhosted.org/packages/85/4b/953118a730ee3b35e28645e0eb4cf9beec5bdbb954e1ac2f5fci!"
43     { url = "https://files.pythonhosted.org/packages/44/9b/56dd530c367c74ae17411027cea4135ca57e1e0583bf5594cee!"
44     { url = "https://files.pythonhosted.org/packages/ce/b0/bcd672edad27ecca7da1f7bb0ce72cd1706a4f2d79ae94990af!"
45     { url = "https://files.pythonhosted.org/packages/80/9e/15cdfcb30a1544a46c9e487a00df331c4672450216538705a9!"
46     { url = "https://files.pythonhosted.org/packages/32/4e/8d7656ccaab3e81e97258b8a9bc5f0c8502513a92fb4ceb0a2c!"
47     { url = "https://files.pythonhosted.org/packages/3c/81/97060281b602ed07f21b12f4ec400eac1f75a2f91fbc829ed8h!"
```




uv sync

가상환경을 uv.lock 파일과 **정확히 일치(100% 동일)**시키는 명령어입니다.

uv.lock 파일에 있는데 설치가 되지 않은 것



설치

프로젝트의 의존성으로 선언되지 않은 것



삭제

```
elicer@a4838c06204f:~/elice_uv$ uv sync
Using CPython 3.12.9 interpreter at: /usr/local/bin/python3
Creating virtual environment at: .venv
Resolved 6 packages in 1ms
Installed 4 packages in 144ms
+ numpy==2.5.0
+ pandas==3.0.4
+ python-dateutil==2.9.0.post0
+ six==1.17.0
```

```
▼ ELICE_UV
> .venv
🔗 main.py
⚙️ pyproject.toml
📖 README.md
≡ uv.lock
```